МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования



«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №6

по дисциплине «Численные методы решения прикладных задач»

Выполнил:

Студент группы 8МП5Л _____ Шаврин А.П.

Проверил:

К.х.н., доцент (ОНД, ИШПР) _____ Зарубин А.Г.

Томск 2026

Цель

Изучить метод Ньютона с сопряжённым градиентом (Newton-CG) для задачи безусловной минимизации, сравнить его с квазиньютоновскими методами BFGS/L-BFGS и с точным методом Ньютона (Newton-Exact) на примере функции Розенброка. Оценить поведение методов при минимизации одной точки и батча из нескольких независимых точек.

Задание

1. Познакомиться с библиотекой PyTorch Minimize.
2. Проработать пример rosen_minimize.ipynb на Python.
3. Сформулировать выводы.

Выполнение работы

1. Окружение

Установлена библиотека pytorch-minimize. Код приведён на рисунке 1.

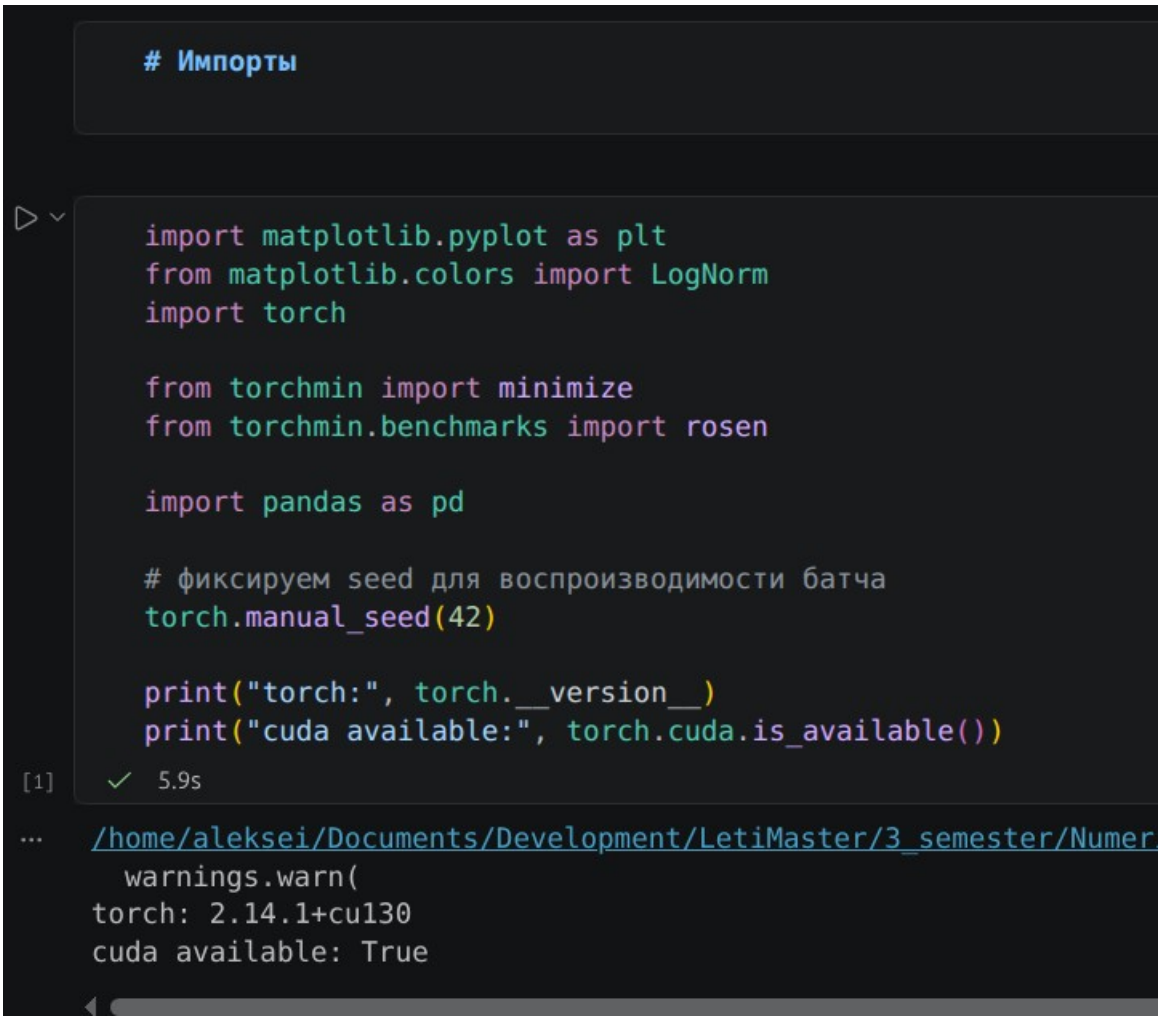


Рисунок 1. Импорты и проверка окружения

2. Функция Розенброка

```python
x, y = torch.meshgrid(
    torch.linspace(-10, 10, 200),
    torch.linspace(-10, 10, 200),
    indexing='ij'
)
xy = torch.stack([x, y], -1)
z = rosen(xy, reduce=False)

fig, ax = plt.subplots(figsize=(6, 5))
c = ax.pcolormesh(x, y, z, shading='auto', cmap='viridis_r',
                  norm=LogNorm(vmin=z.min(), vmax=z.max()))
ax.set_xlabel('X1', fontsize=14)
ax.set_ylabel('X2', fontsize=14, rotation=0)
ax.yaxis.set_label_coords(-0.15, 0.5)
fig.colorbar(c, ax=ax)
plt.title("Rosenbrock function (log-scale)")
plt.show()
```

✓ 0.2s

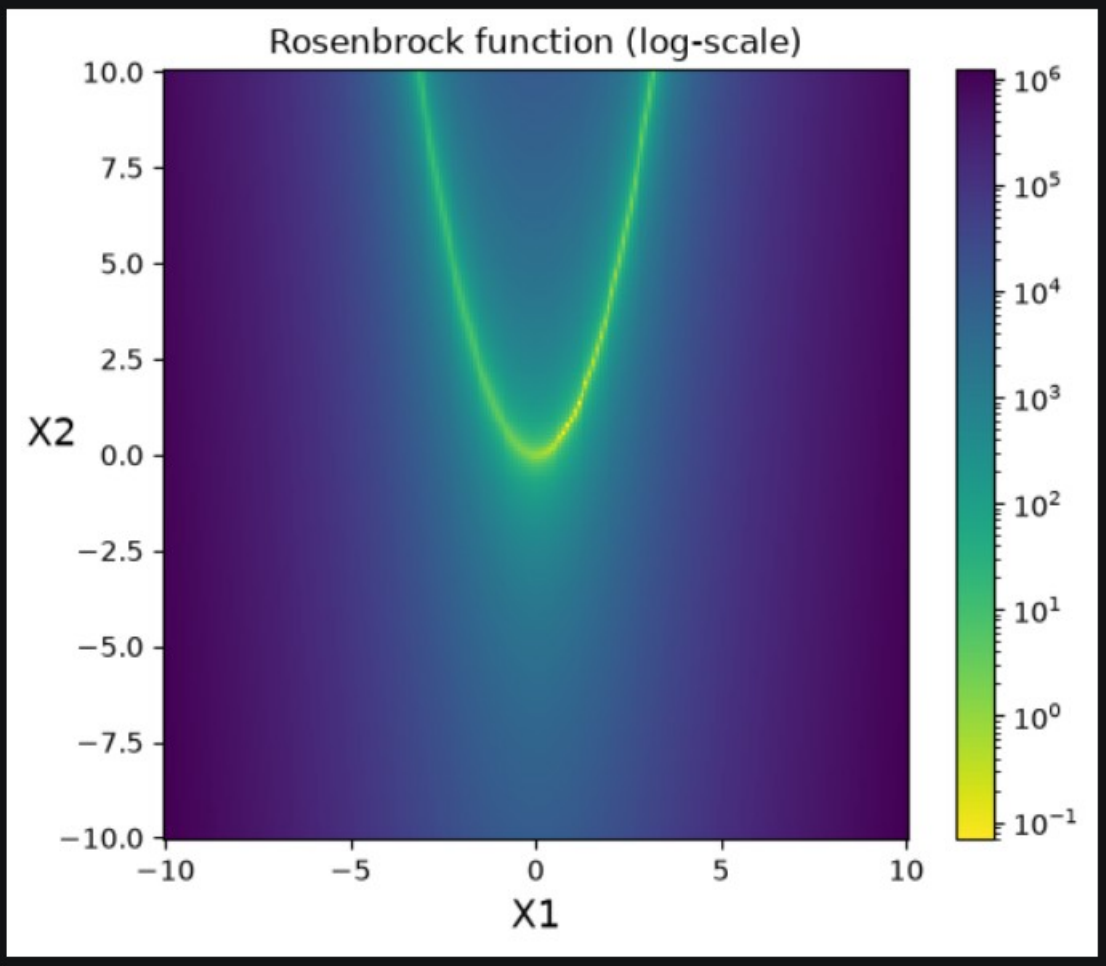


Рисунок 2. Функция Розенброка

3. Минимизация одной точки

```
## BFGS
```

```
x0 = torch.tensor([1., 8.])
print("f(x0) =", rosen(x0).item())

res_bfgs = minimize(
    rosen, x0,
    method='bfgs',
    options=dict(line_search='strong-wolfe'),
    max_iter=50,
    disp=2
)
print("\nfinal x:", res_bfgs.x)
print("final f:", rosen(res_bfgs.x).item())
```

✓ 0.0s

```
f(x0) = 4900.0
initial fval: 4900.0000
iter   1 - fval: 119.3775
iter   2 - fval: 2.7829
iter   3 - fval: 2.7822
iter   4 - fval: 2.7819
iter   5 - fval: 2.7808
iter   6 - fval: 2.7784
iter   7 - fval: 2.7717
iter   8 - fval: 2.7548
iter   9 - fval: 2.7150
iter  10 - fval: 2.6417
iter  11 - fval: 2.5263
iter  12 - fval: 2.2811
iter  13 - fval: 1.8599
iter  14 - fval: 1.5437
iter  15 - fval: 1.1542
iter  16 - fval: 0.9849
iter  17 - fval: 0.8905
iter  18 - fval: 0.6474
iter  19 - fval: 0.5013
iter  20 - fval: 0.3446
iter  21 - fval: 0.2530
iter  22 - fval: 0.2189
iter  23 - fval: 0.1172
...
         Function evaluations: 46

final x: tensor([1.0000, 1.0001])
final f: 2.900435447372729e-09
```

Рисунок 3 — BFGS: код + лог

```python
res_lbfgs = minimize(
    rosen, x0,
    method='l-bfgs',
    options=dict(line_search='strong-wolfe'),
    max_iter=50,
    disp=2
)
print("\nfinal x:", res_lbfgs.x)
print("final f:", rosen(res_lbfgs.x).item())
```

✓ 0.0s

```
initial fval: 4900.0000
iter   1 - fval: 119.3775
iter   2 - fval: 2.7829
iter   3 - fval: 2.7823
iter   4 - fval: 2.7822
iter   5 - fval: 2.7818
iter   6 - fval: 2.7810
iter   7 - fval: 2.7785
iter   8 - fval: 2.7723
iter   9 - fval: 2.7564
iter  10 - fval: 2.7188
iter  11 - fval: 2.6479
iter  12 - fval: 2.5357
iter  13 - fval: 2.3006
iter  14 - fval: 1.8822
iter  15 - fval: 1.5530
iter  16 - fval: 1.1887
iter  17 - fval: 1.0868
iter  18 - fval: 0.9367
iter  19 - fval: 0.6710
iter  20 - fval: 0.5970
iter  21 - fval: 0.4478
iter  22 - fval: 0.3348
iter  23 - fval: 0.2421
iter  24 - fval: 0.2027
...
         Function evaluations: 47

final x: tensor([1.0000, 1.0000])
final f: 1.3984902125230292e-10
```

Рисунок 4 — L-BFGS: код + лог.

```
## Newton-CG
```

```
res_ncg = minimize(
    rosen, x0,
    method='newton-cg',
    options=dict(line_search='strong-wolfe'),
    max_iter=50,
    disp=2
)
print("\nfinal x:", res_ncg.x)
print("final f:", rosen(res_ncg.x).item())
```

✓ 1.0s

```
initial fval: 4900.0000
iter   1 - fval: 6.0505
iter   2 - fval: 2.8156
iter   3 - fval: 2.8144
iter   4 - fval: 2.3266
iter   5 - fval: 2.1088
iter   6 - fval: 1.7060
iter   7 - fval: 1.5851
iter   8 - fval: 1.2548
iter   9 - fval: 1.1625
iter  10 - fval: 0.8967
iter  11 - fval: 0.8249
iter  12 - fval: 0.6160
iter  13 - fval: 0.5591
iter  14 - fval: 0.4051
iter  15 - fval: 0.3299
iter  16 - fval: 0.2217
iter  17 - fval: 0.1886
iter  18 - fval: 0.1167
iter  19 - fval: 0.0987
iter  20 - fval: 0.0543
iter  21 - fval: 0.0442
iter  22 - fval: 0.0210
iter  23 - fval: 0.0118
iter  24 - fval: 0.0035
...
         CG iterations: 41

final x: tensor([1.0000, 1.0000])
final f: 7.817391178832622e-11
```

Рисунок 5 — Newton-CG: код + лог

```
## Newton-Exact (с регуляризацией Тихонова)
```

```
res_ne = minimize(
    rosen, x0,
    method='newton-exact',
    options=dict(line_search='strong-wolfe', tikhonov=1e-4),
    max_iter=50,
    disp=2
)
print("\nfinal x:", res_ne.x)
print("final f:", rosen(res_ne.x).item())
```

✓ 0.1s

```
initial fval: 4900.0000
iter   1 - fval: 3.3641 - info: 1
iter   2 - fval: 2.8111 - info: 2
iter   3 - fval: 2.3209 - info: 0
iter   4 - fval: 1.7723 - info: 0
iter   5 - fval: 1.4215 - info: 0
iter   6 - fval: 0.9833 - info: 0
iter   7 - fval: 0.7774 - info: 0
iter   8 - fval: 0.5245 - info: 0
iter   9 - fval: 0.3976 - info: 0
iter  10 - fval: 0.2116 - info: 0
iter  11 - fval: 0.1417 - info: 0
iter  12 - fval: 0.0715 - info: 0
iter  13 - fval: 0.0381 - info: 0
iter  14 - fval: 0.0113 - info: 0
iter  15 - fval: 0.0046 - info: 0
iter  16 - fval: 0.0002 - info: 0
iter  17 - fval: 0.0000 - info: 0
iter  18 - fval: 0.0000 - info: 0
iter  19 - fval: 0.0000 - info: 0
iter  20 - fval: 0.0000 - info: 0
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 20
         Function evaluations: 58

final x: tensor([1., 1.])
final f: 0.0
```

Рисунок 6 — Newton-Exact: код + лог (видна колонка info).

Newton-Exact достиг точного нуля (f=0, x=(1,1)) за 20 итераций. В первых двух итерациях в логе info = 1 и info = 2 — Cholesky не проходил, метод

работал в fallback-режиме (steepest descent / LU). С третьей итерации info = 0 — Cholesky стабилен. В эталонном rosen_minimize.ipynb этот же метод сходится за 2 итерации; различие связано с реализацией Cholesky/fallback в конкретной версии torchmin, а не с математикой задачи. Финальный результат совпадает: f=0, x=(1,1).

Newton-CG даёт почти ту же точность (10^−11) за 29 итераций, используя только Hessian-vector product. Память — O(n) вместо O(n^2), что делает его применимым к задачам большой размерности.

BFGS и L-BFGS не используют вторых производных и потому сходятся медленнее (36 и 38 итераций). L-BFGS оказался немного точнее BFGS.

4. Минимизация батча из 4 точек

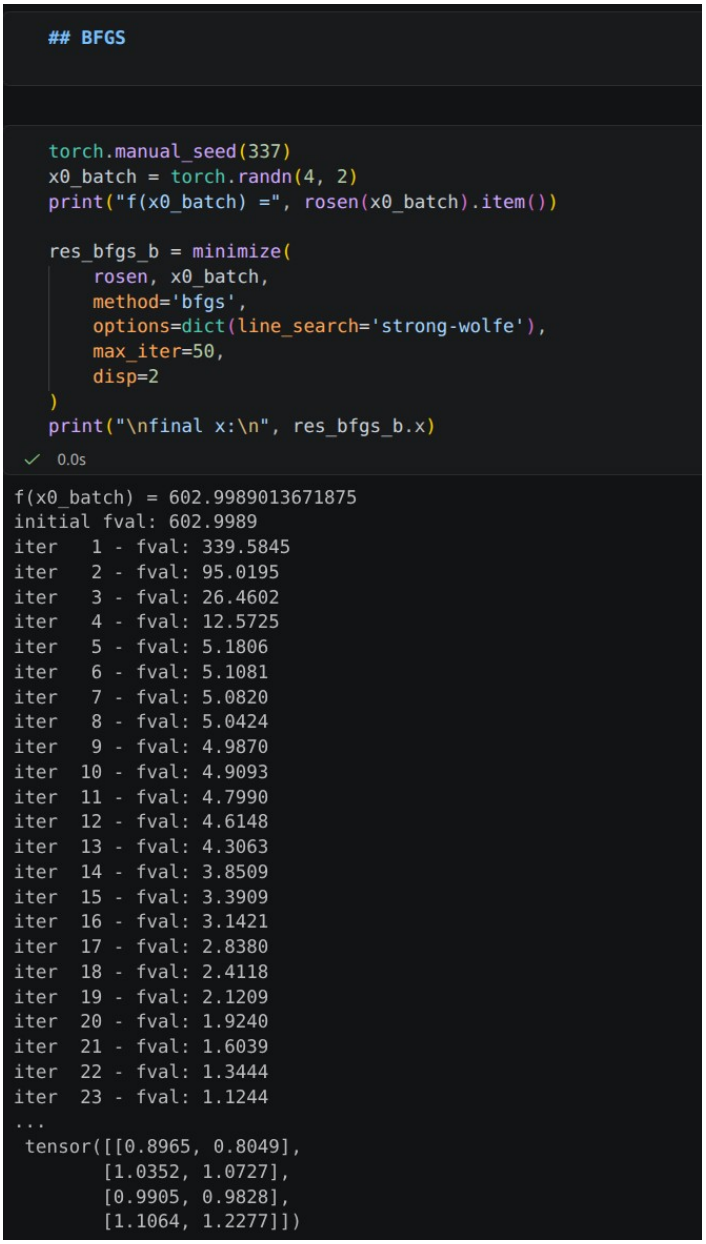


Рисунок 7 — BFGS на батче: код + лог.

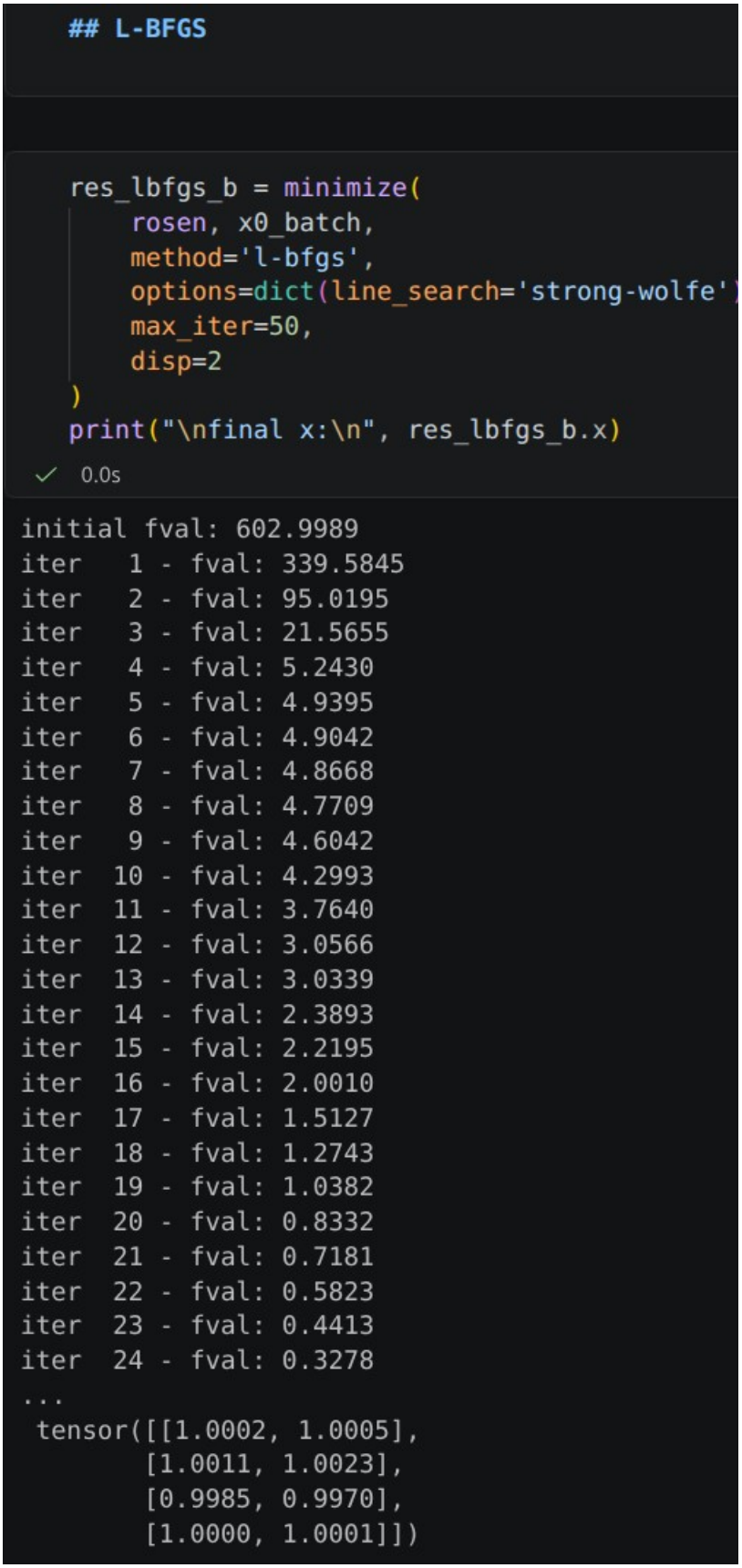


Рисунок 8 — L-BFGS на батче: код + лог.

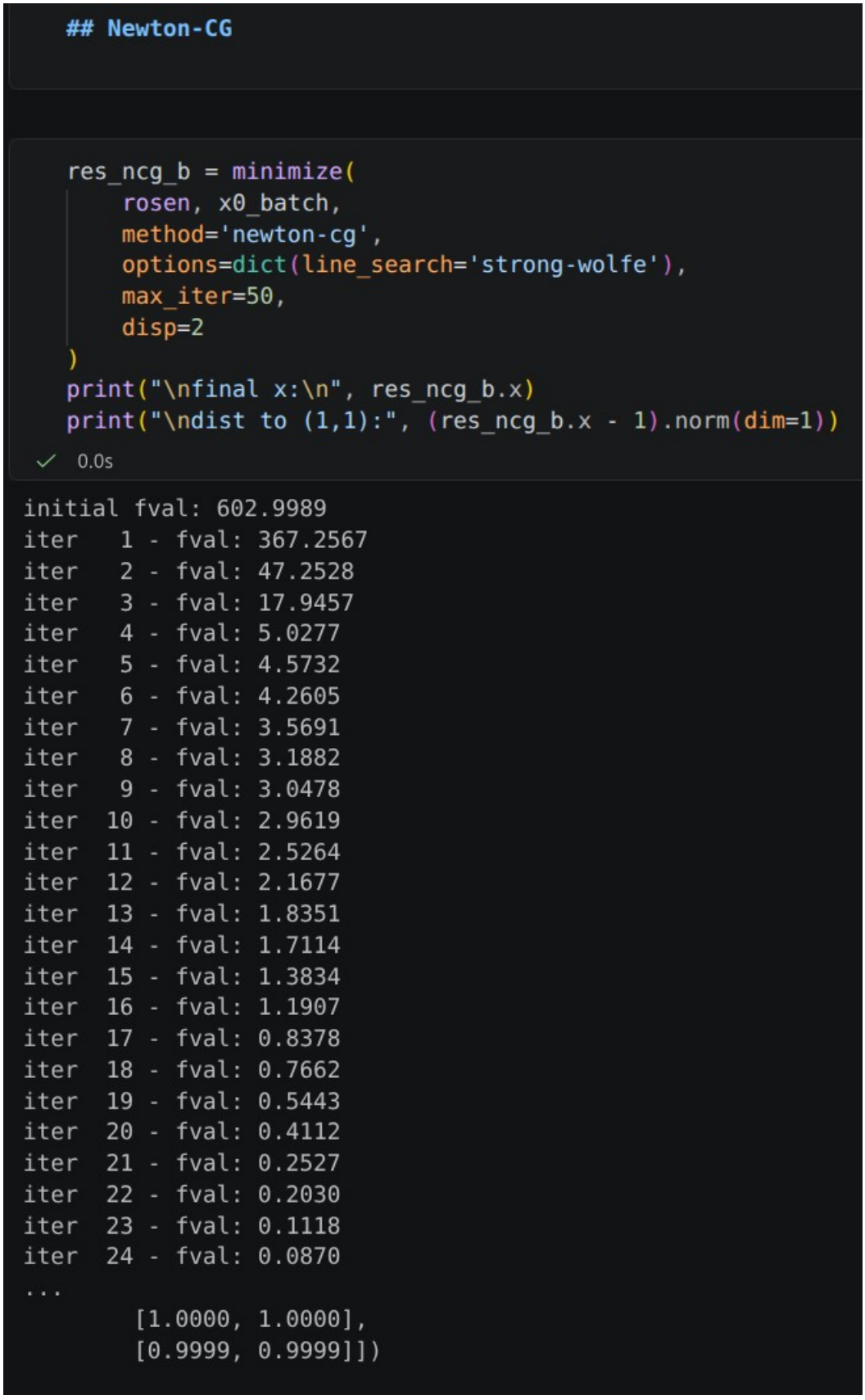


Рисунок 9 — Newton-CG на батче: код + лог + dist to (1,1).

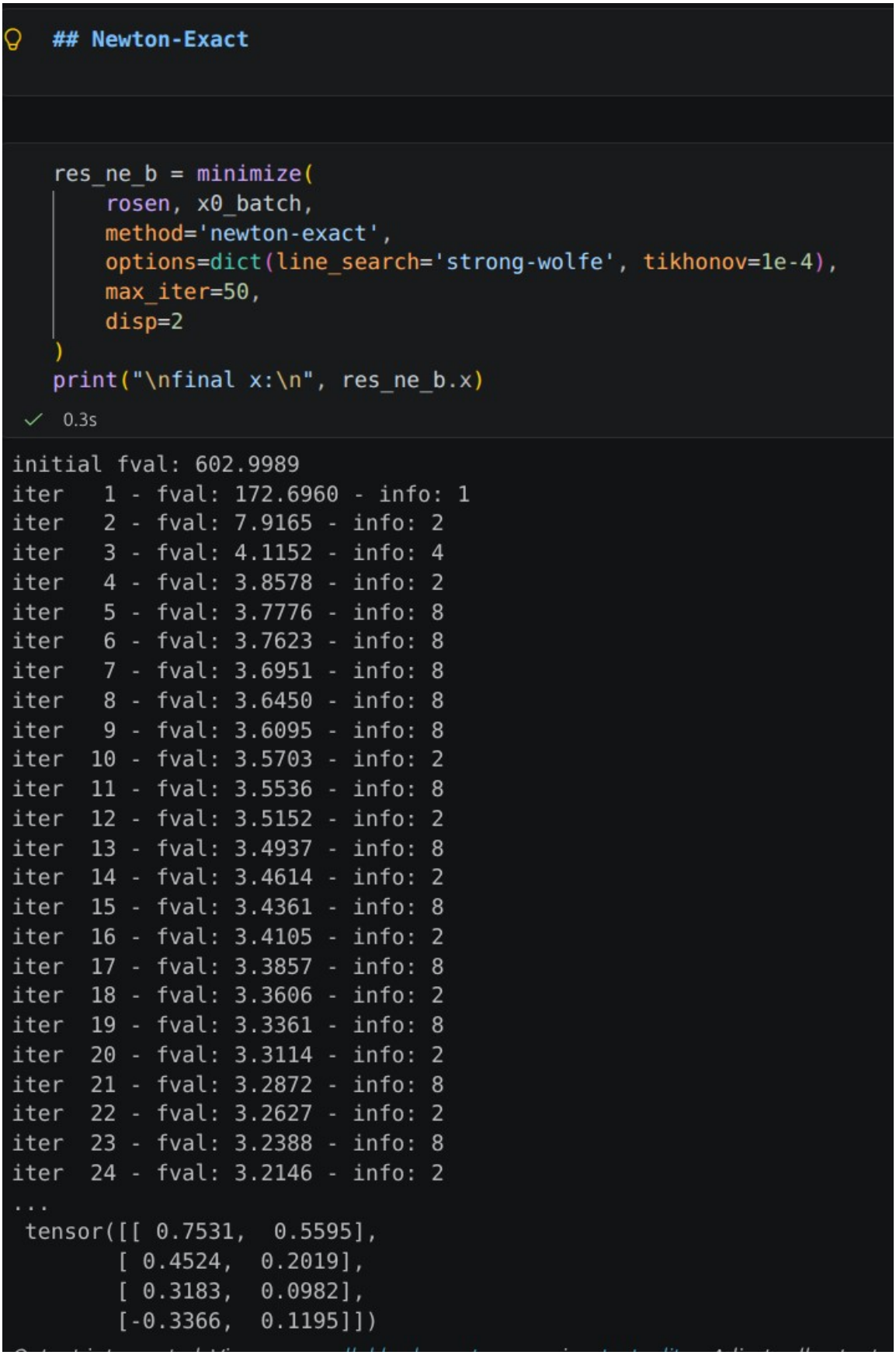


Рисунок 10 — Newton-Exact на батче: код + лог.

Newton-CG — единственный метод, который корректно решил задачу для всех 4 точек за 32 итерации. Причина в том, что внутренний CG решает подзадачу Hh=−g для каждой точки отдельно, тогда как line search даёт лишь общий масштаб шага. Это делает Newton-CG устойчивым к гетерогенности батча, где точки требуют разного шага.

BFGS не сошёлся (f≈2.5·10^−2, три из четырёх точек далеки от (1,1)). Виноват общий шаг line search: компромиссный αk не подходит ни одной точке достаточно хорошо, и метод «застревает» на полке около f≈0.05.

L-BFGS почти сошёлся (f≈3.5·10^−6) — благодаря ограниченной памяти, которая не даёт накапливать плохие пары (sk,yk) от разных точек. Но до точности Newton-CG он всё равно не дотягивает.

Newton-Exact не сошёлся (f≈2.62): колонка info в логе постоянно принимает значения 2 и 8 — Cholesky падает на каждой итерации, метод работает в fallback и почти не движется. В эталонном rosen_minimize.ipynb этот же метод на том же батче сходится за 20 итераций, значит поведение зависит от версии библиотеки torchmin, а не от свойств гессиана.

Практический вывод. На батче независимых точек Newton-Exact в данной версии библиотеки оказался неработоспособен из-за постоянных падений Cholesky. Newton-CG — компромисс: он использует кривизну, но не требует полного гессиана, потому что CG решает систему итерационно. Именно поэтому на батче он — единственный надёжный вариант из рассмотренных.

5. Траектории

```python
def run_with_trace(method, x0, **opts):
    traj = [x0.detach().clone()]
    def cb(x):
        traj.append(x.detach().clone())
    res = minimize(rosen, x0, method=method, options=opts,
                   max_iter=50, disp=0, callback=cb)
    return res, torch.stack(traj)

x0 = torch.tensor([1., 8.])
_, traj_ncg = run_with_trace('newton-cg', x0, line_search='strong-wolfe')
_, traj_ne  = run_with_trace('newton-exact', x0,
                             line_search='strong-wolfe', tikhonov=1e-4)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# --- левый график: Newton-CG ---
ax = axes[0]
ax.pcolormesh(x, y, z, shading='auto', cmap='viridis_r',
              norm=LogNorm(vmin=z.min(), vmax=z.max()))
ax.plot(traj_ncg[:, 0], traj_ncg[:, 1], 'o-', color='orange',
        label='Newton-CG', lw=2, markersize=6)
ax.plot(1, 1, 'w*', markersize=18, label='minimum (1,1)')
ax.set_xlabel('X1'); ax.set_ylabel('X2')
ax.set_title("Newton-CG trajectory")
ax.legend()

# --- правый график: Newton-Exact ---
ax = axes[1]
ax.pcolormesh(x, y, z, shading='auto', cmap='viridis_r',
              norm=LogNorm(vmin=z.min(), vmax=z.max()))
ax.plot(traj_ne[:, 0], traj_ne[:, 1], 's-', color='red',
        label='Newton-Exact', lw=2, markersize=6)
ax.plot(1, 1, 'w*', markersize=18, label='minimum (1,1)')
ax.set_xlabel('X1'); ax.set_ylabel('X2')
ax.set_title("Newton-Exact trajectory")
ax.legend()

plt.tight_layout()
plt.show()
```

✓ 0.3s

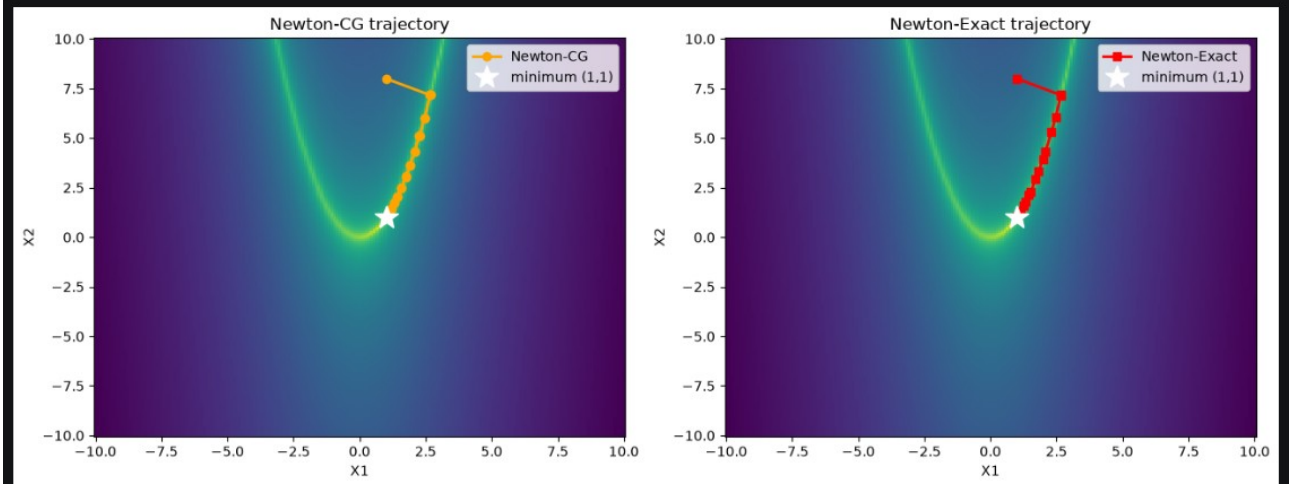
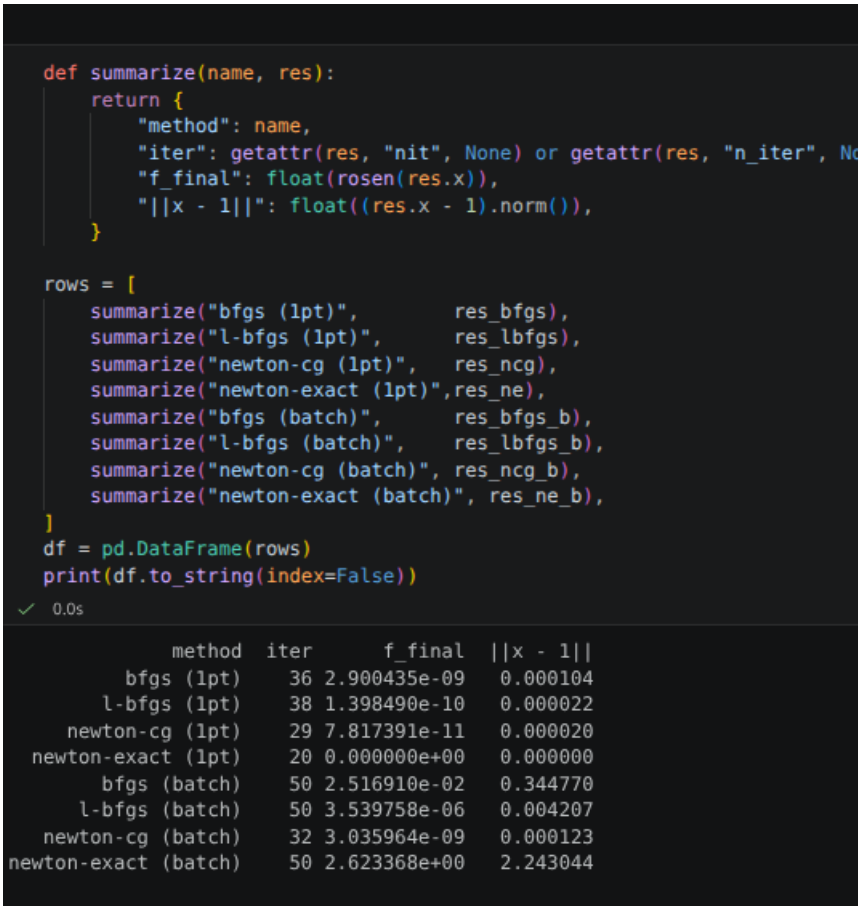


Рисунок 11 — траектории Newton-CG (слева) и Newton-Exact (справа) из точки (1,8) на фоне рельефа.

6. Сводная таблица



```
def summarize(name, res):
    return {
        "method": name,
        "iter": getattr(res, "nit", None) or getattr(res, "n_iter", No
        "f_final": float(rosen(res.x)),
        "||x - 1||": float((res.x - 1).norm()),
    }

rows = [
    summarize("bfgs (1pt)",         res_bfgs),
    summarize("l-bfgs (1pt)",       res_lbfgs),
    summarize("newton-cg (1pt)",    res_ncg),
    summarize("newton-exact (1pt)",res_ne),
    summarize("bfgs (batch)",       res_bfgs_b),
    summarize("l-bfgs (batch)",     res_lbfgs_b),
    summarize("newton-cg (batch)", res_ncg_b),
    summarize("newton-exact (batch)", res_ne_b),
]
df = pd.DataFrame(rows)
print(df.to_string(index=False))
```

✓ 0.0s

```
            method  iter      f_final  ||x - 1||
         bfgs (1pt)    36 2.900435e-09   0.000104
       l-bfgs (1pt)    38 1.398490e-10   0.000022
    newton-cg (1pt)    29 7.817391e-11   0.000020
 newton-exact (1pt)    20 0.000000e+00   0.000000
       bfgs (batch)    50 2.516910e-02   0.344770
     l-bfgs (batch)    50 3.539758e-06   0.004207
  newton-cg (batch)    32 3.035964e-09   0.000123
newton-exact (batch)    50 2.623368e+00   2.243044
```

Рисунок 12 — итоговый DataFrame со всеми 8 запусками.

Вывод

В ходе работы изучен метод Ньютона с сопряжённым градиентом (Newton-CG) и проведено его сравнение с BFGS, L-BFGS и Newton-Exact на функции Розенброка. Использована библиотека pytorch-minimize, все градиенты и Hessian-vector products считаются автоматически через autograd.

По одиночной точке x0=(1,8):

- Все четыре метода сошлись в минимум (1,1) с f≤10^−9.
- Newton-Exact достигает точного нуля — подтверждается квадратичная сходимость метода Ньютона вблизи минимума. Цена — полный гессиан и Cholesky на каждой итерации (O(n3)).
- Newton-CG даёт почти ту же точность (f≈7.8·10^−11) за 29 итераций, используя только Hessian-vector product. Память — O(n) вместо O(n^2).
- BFGS и L-BFGS обходятся без вторых производных, но сходятся медленнее (36 и 38 итераций).

По батчу из 4 точек:

- Newton-CG — единственный метод, который корректно решил задачу для всех четырёх точек: 32 итерации, все точки в (1,1) с точностью 10^−4.
- BFGS не сошёлся (f≈2.5·10^−2): общий шаг line search не подходит точкам из разных частей долины.
- L-BFGS почти сошёлся (f≈3.5·10^−6), но точности Newton-CG не достиг.
- Newton-Exact не сошёлся (f≈2.62): Cholesky постоянно падает, info чередуется между 2 и 8. В эталонном rosen_minimize.ipynb этот же метод на том же батче сходится за 20 итераций — поведение зависит от версии библиотеки.

Общий вывод. Newton-CG — оптимальный компромисс между точностью метода второго порядка и сложностью метода первого порядка. Он не хранит гессиан, использует только HVP, устойчив к гетерогенности батча и единственный из рассмотренных методов корректно работает на батче.